# CIVICLENS – CONSTITUENCY ACCOUNTABILITY & CIVIC TRANSPARENCY

**GE23627 - DESIGN THINKING AND INNOVATION PROJECT**

*Submitted by*


**Durgaprasad V – 241701014**

**Sanjaykumar R – 241701049**


*in partial fulfillment for the award of the degree of*

**BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND DESIGN**







**RAJALAKSHMI ENGINEERING COLLEGE**

**OCTOBER 2026**

**RAJALAKSHMI ENGINEERING COLLEGE**

**ANNA UNIVERSITY, CHENNAI**

## BONAFIDE CERTIFICATE

Certified that this Report titled "CIVICLENS – CONSTITUENCY ACCOUNTABILITY & CIVIC TRANSPARENCY" is the bonafide work of Durgaprasad V (241701014) and Sanjaykumar R (241701049) who carried out the work under the guidance of the Department of Computer Science and Design. Certified further that to the best of our knowledge the work reported herein is prepared for the academic project and does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Certified further that the project supports SDG 9 – Industry, Innovation and Infrastructure, SDG 11 – Sustainable Cities and Communities, SDG 16 – Peace, Justice and Strong Institutions, and SDG 17 – Partnerships for the Goals through transparent civic service delivery, accountable workflows and accessible public information.

| Signature of the HOD | Signature of the Supervisor |
|---|---|
| Dr. S. Vinodh Kumar<br>Professor,The Head of the Department<br>Department of Computer Science and Design<br>Rajalakshmi Engineering College<br>Chennai | Dr. P. Revathy<br>Professor<br>Department of Computer Science and Design<br>Rajalakshmi Engineering College<br>Chennai |

Submitted for the Phase I Project Viva-Voce examination held on _____.

INTERNAL EXAMINER EXTERNAL EXAMINER

# ABSTRACT

Citizens frequently encounter potholes, garbage accumulation, drainage failures, water-supply interruptions, streetlight faults and damaged public facilities. Existing grievance channels may accept complaints, yet the journey after submission can remain fragmented: citizens may not know which department is responsible, how work is progressing, why a case is delayed or whether a complaint marked resolved has actually been fixed. CivicLens is a citizen-centric civic accountability and transparency platform designed to connect reporting with jurisdiction-aware authority routing, complaint timelines, field updates and citizen verification. The platform also identifies recurring issue locations and presents constituency-level analytics alongside public development projects and expenditure records when reliable sources are available. Its central design principle separates departmental execution from elected-representative oversight, reducing the risk of attributing work to an authority that does not control it. The proposed system includes citizen, department officer, field worker, representative office, administrator and public-viewer roles. A free-first architecture uses React or Next.js, Tailwind CSS, Supabase/PostgreSQL, OpenStreetMap with Leaflet, and charting libraries; AI assistance remains optional, with rule-based fallback. This Phase I report documents the problem, related concepts, system design, requirements, modules, data flow, feasibility, testing strategy and future development plan. Prototype data must be clearly labelled, and public financial claims must be supported by traceable sources.

# ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. S. VINODHKUMAR, Ph.D.,** Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work .We convey our sincere and deepest gratitude to our internal guide **,DR.P.REVATHY.,** Department of Computer Science and Design , for her valuable guidance throughout the course of the project. We are very glad to thank our Project Coordinator, **Dr.S.P. KARUPPIAH, Ph.D.,** Department of Computer Science and Design, for his valuable guidance and useful tips during our reviews to build our project.

**Durgaprasad V (241701014)**

**Sanjaykumar R (241701049)**

# TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | ACKNOWLEDGEMENT | iii |
| | ABSTRACT | iv |
| | LIST OF TABLES | v |
| | LIST OF FIGURES | vi |
| | LIST OF ABBREVIATIONS | vii |
| **1.** | **INTRODUCTION** | **12** |
| | 1.1 GENERAL | 12 |
| | 1.2 BACKGROUND | 12 |
| | 1.3 PROBLEM STATEMENT | 13 |
| | 1.4 MOTIVATION | 13 |
| | 1.5 OBJECTIVES | 14 |
| | 1.6 SCOPE | 14 |
| | 1.6.1 INCLUDED SCOPE | 14 |
| | 1.6.2 SCOPE LIMITATIONS | 15 |
| | 1.7 EXISTING SYSTEM | 15 |
| | 1.8 LIMITATIONS OF EXISTING SYSTEM | 16 |
| | 1.9 PROPOSED SYSTEM | 17 |
| | 1.10 ADVANTAGES OF PROPOSED SYSTEM | 17 |
| **2.** | **LITERATURE REVIEW** | **19** |
| | 2.1 INTRODUCTION | 19 |

| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | 2.2 REVIEW OF RELATED WORK | 19 |
| | 2.3 COMPARATIVE ANALYSIS | 21 |
| | 2.4 COMPARATIVE ANALYSIS OF THE PROPOSED SYSTEM | 22 |
| | 2.5 RESEARCH GAP | 23 |
| **3.** | **THEORETICAL BACKGROUND AND SYSTEM DESIGN** | **24** |
| | 3.1 GENERAL | 24 |
| | 3.2 CIVIC ISSUE DATA ACQUISITION | 24 |
| | 3.3 REPORT VALIDATION AND NORMALIZATION | 24 |
| | 3.4 ISSUE CLASSIFICATION AND ROUTING | 24 |
| | 3.5 GEOGRAPHIC DATA AND JURISDICTION | 25 |
| | 3.6 ISSUE PRIORITY AND CATEGORY RULES | 25 |
| | 3.7 JURISDICTION RULE ENGINE | 26 |
| | 3.8 OPENSTREETMAP AND LEAFLET | 26 |
| | 3.9 MAP-BASED ISSUE TRACKING | 26 |
| | 3.10 LOCATION MAPPING AND CONTROL | 26 |
| | 3.11 EVIDENCE AND REPORT ACQUISITION | 26 |
| | 3.12 TEXT NORMALIZATION | 27 |
| | 3.13 OPTIONAL CIVIC ASSISTANT | 27 |
| | 3.14 OPTIONAL CLASSIFICATION MODELS | 27 |
| | 3.15 ISSUE CLASSIFICATION AND ROUTING | 27 |
| | 3.16 COMPLAINT WORKFLOW LAYER | 28 |
| | 3.17 SYSTEM ARCHITECTURE | 28 |
| | 3.18 DATA FLOW DIAGRAM | 29 |

**CHAPTER NO.** **TITLE** **PAGE NO.**


3.18.1 LEVEL 0 30

3.18.2 LEVEL 1 30

3.18.3 LEVEL 2 31

3.19 USE CASE DIAGRAM 32

3.20 ACTIVITY DIAGRAM 32

3.21 OVERALL SYSTEM DIAGRAM 33

3.22 CORE CONCEPTUAL CLASSES 34

**4. METHODOLOGY AND PROJECT DESCRIPTION** **36**

4.1 GENERAL 36

4.2 REQUIREMENT ANALYSIS 36

4.3 FUNCTIONAL REQUIREMENTS 37

4.4 NON-FUNCTIONAL REQUIREMENTS 37

4.5 TECHNOLOGY SELECTION 38

4.6 SYSTEM MODULES 39

4.6.1 CIVIC ISSUE REPORTING MODULE 40

4.6.2 AUTHORITY ROUTING AND TRACKING MODULE 40

4.6.3 VERIFICATION AND TRANSPARENCY MODULE 40

4.6.4 WORKFLOW AND ANALYTICS MODULE 41

4.7 COMPLAINT STATUS AND ACTION MAPPING 41

4.8 MODULE INTEGRATION 42

4.9 SYSTEM CONFIGURATION 43

4.10 FEASIBILITY STUDY 44

**CHAPTER NO.** **TITLE** **PAGE NO.**


4.10.1 ECONOMICAL FEASIBILITY 45

4.10.2 TECHNICAL FEASIBILITY 45

4.10.3 SOCIAL FEASIBILITY 45

4.11 TESTING METHODOLOGY 45

4.11.1 UNIT TESTING 46

4.11.2 INTEGRATION TESTING 46

4.11.3 FUNCTIONAL TESTING 46

4.11.4 SYSTEM TESTING 47

4.11.5 WHITE BOX AND BLACK BOX TESTING 47

4.12 INITIAL RESULTS 48

4.13 CHALLENGES 48

**5. CONCLUSIONS AND FUTURE ENHANCEMENTS FOR PHASE II** **50**

5.1 CONCLUSION 50

5.2 CURRENT PHASE-I STATUS 50

5.3 LIMITATIONS 51

5.4 PHASE-II OBJECTIVES 52

5.5 FUTURE ENHANCEMENTS 53

5.6 PHASE-II WORK SCHEDULE 54

REFERENCES 55

CO–SDG RELEVANCE RECORD 56

PROGRAM OUTCOMES 57

PROGRAM SPECIFIC OUTCOMES 58

CO–PO–PSO MAPPING 59

# LIST OF TABLES


| Table No. | Title | Page No. |
|---|---|---|
| Table 1.1 | Limitations of existing grievance workflows | 16 |
| Table 2.1 | Literature and civic-service comparison | 21 |
| Table 2.2 | Comparative analysis of civic-service approaches | 22 |
| Table 3.1 | Core CivicLens data entities | 35 |
| Table 4.1 | Functional requirements | 37 |
| Table 4.2 | Non-functional requirements | 38 |
| Table 4.3 | Technology stack | 39 |
| Table 4.4 | Technology selection rationale | 39 |
| Table 4.5 | System module descriptions | 40 |
| Table 4.6 | Mapping of user actions to workflow outcomes | 41 |
| Table 4.7 | Hardware configuration | 43 |
| Table 4.8 | Software configuration | 44 |
| Table 4.9 | Prototype acceptance test cases | 47 |
| Table 5.1 | Future enhancements | 54 |

# LIST OF FIGURES


| Figure No. | Title | Page No. |
|---|---|---|
| Figure 3.1 | CivicLens system architecture | 29 |
| Figure 3.2 | Data flow diagram – Level 0 | 30 |
| Figure 3.3 | Data flow diagram – Level 1 | 31 |
| Figure 3.4 | Data flow diagram – Level 2 | 31 |
| Figure 3.5 | Use case diagram | 32 |
| Figure 3.6 | Activity diagram | 33 |
| Figure 3.7 | Overall system diagram | 34 |

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| GIS | Geographic Information System |
| UI | User Interface |
| UX | User Experience |
| RBAC | Role-Based Access Control |
| SLA | Service-Level Agreement |
| CRUD | Create, Read, Update and Delete |
| DB | Database |
| FR | Functional Requirement |
| NFR | Non-Functional Requirement |
| ID | Identifier |
| OSM | OpenStreetMap |
| PO | Program Outcome |
| PSO | Program Specific Outcome |
| SDG | Sustainable Development Goal |
| SQL | Structured Query Language |
| UML | Unified Modeling Language |
| URL | Uniform Resource Locator |
| CSV | Comma-Separated Values |
| API | Application Programming Interface |
| PII | Personally Identifiable Information |

# CHAPTER 1

# INTRODUCTION

## 1.1 General

Public services are experienced locally, through roads, drainage, sanitation, water supply, streetlights and shared facilities. When these services fail, citizens need more than a form that accepts a complaint. They need to understand where the issue was sent, who is expected to act, what has happened so far and whether the reported problem has been resolved. CivicLens addresses this service-design challenge by treating a civic complaint as a traceable journey rather than an isolated submission. The platform brings reporting, authority routing, work updates, citizen verification and public transparency into one connected experience.

The proposed platform begins with a citizen report containing a category, description, location and optional evidence. The system checks the information and uses category and jurisdiction rules to identify a likely responsible department. Officers review and assign work, field workers record inspection or completion updates, and the citizen is asked to verify the claimed outcome. Unresolved or disputed cases can be reopened or escalated. Aggregated complaint records support maps, recurring-issue analytics and constituency dashboards. Public project and expenditure details are shown only when a source or an explicitly labelled demonstration record is available.

## 1.2 Background

Civic grievance processes can become difficult to follow when responsibility is unclear and information is spread across separate channels. Citizens may submit duplicate complaints because they cannot see existing reports, while departments may receive incomplete descriptions or inaccurate locations. A case can be marked resolved without a clear mechanism for affected residents to confirm the result. Repeated failures at the same location may therefore appear as unrelated incidents, and project

expenditure may be difficult to connect with visible service outcomes. CivicLens frames these gaps as a connected accountability problem.

The project proposes a unified workflow: citizen reporting → evidence and location checks → jurisdiction-based routing → officer assignment → field action and updates → citizen verification → reopening or escalation where needed → public analytics. The system is designed as a prototype and does not assume live integration with government databases. Administrative boundaries, department responsibilities and public financial records must be validated for the selected pilot geography.

### 1.3 Problem Statement

Citizens face recurring civic problems such as potholes, overflowing drains, garbage accumulation, water-supply issues and non-functional streetlights. They may not know the responsible authority, receive little visibility into complaint progress, or lack a reliable way to challenge a claimed resolution. Authorities also need a consolidated view of duplicates, overdue cases and repeated problems. A conventional complaint-registration interface alone does not provide the complete traceability needed to connect a report to accountable action and verified outcomes.

### 1.4 Motivation

The motivation for CivicLens comes from two related observations. First, a complaint is useful only when the citizen can follow its progress and understand who is responsible for the next step. Second, recurring issues often indicate a wider infrastructure or service problem that cannot be understood by viewing each complaint in isolation. Connecting individual reports to authority workflows, geographic patterns and verified project information can help citizens understand outcomes and help administrators prioritize work.

CivicLens aims to make the civic-resolution journey easier to understand without treating transparency as a popularity contest. It separates department execution from representative oversight, requires citizen feedback after a resolution claim, and makes the provenance of public information visible. The design prioritizes accessibility, privacy, auditable status changes and a limited pilot scope before expansion.

## 1.5 Objectives

1. To design an integrated civic accountability platform that allows citizens to report local problems, follow responsible authorities, verify resolutions and explore constituency-level service information.
2. To collect structured issue details, including category, description, location, impact and optional photo evidence, with a unique complaint identifier.
3. To apply category and jurisdiction rules to recommend the likely responsible authority and expose the routing decision to the citizen.
4. To maintain a timestamped complaint timeline covering submission, review, assignment, field updates, resolution claims and verification.
5. To provide citizen resolution choices—Completely Resolved, Partially Resolved or Not Resolved—with a path for reopening or escalation.
6. To support officer and field-worker workflows for reviewing, assigning, inspecting, updating and documenting civic work.
7. To present public issue maps, recurring-problem analytics and constituency dashboards based on traceable complaint records.
8. To connect civic issues with public development projects and expenditure information only when a reliable source or clearly labelled sample record is available.

## 1.6 Scope

### 1.6.1 Included Scope

1. Live video capture from a standard location and evidence input and live audio capture from a citizen report input
2. Citizen registration, sign-in and role-based access for citizens, department officers, field workers, representative offices, administrators and public viewers.
3. Civic issue reporting with category, title, description, location, impact details and optional photo/video evidence.
4. Category- and jurisdiction-based routing, authority review, assignment, reassignment and an auditable complaint timeline.

5. Officer and field-worker dashboards for work queues, inspection notes, status changes and evidence uploads.
6. Citizen verification of claimed resolutions, with reopen and escalation paths for disputed outcomes.
7. Public civic map with issue markers, filters, recurring-location clusters and privacy-safe summaries.
8. Constituency dashboard with issue counts, status distributions, overdue cases, categories and response trends.
9. Public project transparency records with responsible agency, available funding, progress, expenditure and source details.
10. Notifications and a simple Civic Assistant for explaining reporting steps and complaint status; AI is optional and must not make final administrative decisions.

### 1.6.2 Scope Limitations

1. CivicLens is an academic prototype and is not a replacement for official grievance portals or emergency services.
2. Routing quality depends on accurate location, configured jurisdiction boundaries and up-to-date department mappings.
3. Public project and expenditure details depend on the availability and reliability of official source data; missing data must be shown explicitly.
4. The initial version focuses on a limited set of civic categories and a single pilot geography rather than every department or constituency.
5. Analytics are descriptive and depend on the completeness of submitted records; they must not be interpreted as proof of causation or overall political performance.
6. Advanced AI prediction, government-system integrations and large-scale deployment are future possibilities, not assumed completed features.

## 1.7 Existing System

Existing grievance channels provide important mechanisms for registering public complaints, but their user journeys may be distributed across websites, telephone lines,

office visits and department-specific applications. Separate tools may handle issue submission, field operations, project information and public reporting. CivicLens proposes to bring the citizen-facing journey and a privacy-safe public view together while preserving the authority of the responsible department.

**Table 1.1 Limitations of existing grievance workflows**

| Existing approach | Primary capability |
|---|---|
| Department-specific grievance portals | Accept and track complaints within a defined service channel |
| Telephone / office-based reporting | Provide access for citizens who cannot use a web form |
| Public maps and open-data dashboards | Visualize location-based issues or publish selected public records |
| CivicLens proposed platform | Connect reporting, authority routing, tracking, verification and public transparency |

## 1.8 Limitations of Existing System

Existing approaches often expose only a portion of the journey. A submission receipt does not necessarily explain jurisdiction, assignment, field action, expected timelines or verification. When a case is closed by an authority, a citizen may not have a clear, structured way to report that the problem remains. CivicLens therefore treats the timeline and verification record as core system objects rather than optional feedback.

A second limitation is fragmented information. Similar issues may be reported repeatedly in one location without a shared hotspot view, and project announcements may be presented separately from complaints about the related infrastructure. CivicLens links records only when the location, category and source information support a defensible relationship. It also labels sample data and avoids exposing citizens' personal details in public views.

1. Incorrect jurisdiction mapping can send a complaint to the wrong department and delay action.
2. Unstructured reports, missing locations and duplicate submissions can increase the workload of officers and field workers.
3. A department's resolution claim may not reflect the affected citizen's experience unless verification is recorded.

**CivicLens combines structured reporting, rule-based routing, timestamped tracking, officer and field-worker workflows, citizen verification, escalation, public civic analytics and project transparency. Each complaint has a unique ID and a history of important actions. Department officers retain responsibility for execution, while constituency representatives receive an oversight view of recurring and unresolved cases.**

The platform is organised into a citizen interface, authority workflow and public-information layer. The reporting module captures issue details and evidence. A routing service uses category and configured jurisdiction rules to suggest an authority, with manual review when uncertain. The complaint service stores assignments and timeline events; verification and escalation preserve the citizen's response. Analytics aggregate records into maps and dashboards, while a separate project module displays source-labelled development information. Authentication and role permissions restrict access to private records.

Citizen Web App → Issue and Evidence Service → Jurisdiction / Authority Routing → Officer Assignment → Field Worker Updates → Complaint Timeline → Citizen Verification → Reopen / Escalate → Civic Analytics and Public Transparency

## 1.10 Advantages of Proposed System

1. Offers one visible journey from complaint submission to citizen-verified outcome.
2. Makes the responsible department and routing status visible rather than leaving ownership unclear.
3. Records timestamped actions, field evidence and verification so disputed closures can be reviewed.
4. Highlights recurring locations and categories to support better workload prioritization.
5. Separates departmental execution from representative oversight to avoid misleading accountability claims.

6. Uses a free-first technology stack and avoids making paid APIs or AI services mandatory.
7. Provides source labels and privacy-safe public dashboards while allowing advanced features to be added in phases.

# CHAPTER 2

# LITERATURE REVIEW

## 2.1 Introduction

The literature and practice review for CivicLens focuses on electronic grievance redress, citizen participation, geographic information systems, public-service performance measurement and open-data transparency. These areas show why a digital complaint form alone is insufficient: a usable platform also needs clear ownership, status visibility, evidence, accessible participation and trustworthy data. The project uses these ideas as design foundations rather than claiming a new administrative policy or a validated impact result.

## 2.2 Review of Related Work

### [1] Vidula Srivatsa et al. (2026)

Public grievance systems provide structured channels for citizens to submit service-related complaints. For CivicLens, the relevant design lesson is that submission should generate a traceable record and a clear next step. A prototype must not imply that it is officially integrated with any government portal unless such access is actually available.

### [2] Swapnil Sharma et al.

Citizen-facing digital services work best when forms use clear language, minimize unnecessary typing and explain what happens after submission. CivicLens applies this principle through a small set of categories, optional evidence, location confirmation and a visible timeline. Accessibility and regional-language support are included as design requirements for the pilot.

### [3] Sukumar Ganapati (2011)

Geographic information systems can reveal spatial concentrations that are hard to see in individual complaint records. A civic map can help identify repeated pothole,

drainage or streetlight reports within a limited area. However, map clusters are indicators for review rather than proof of a root cause, and public maps should not reveal private citizen information.

**[4] Zarin T. Khan and Peter Allan Johnson**

Service-performance measurement requires indicators that are defined, traceable and interpreted within their limits. CivicLens distinguishes departmental resolution performance from representative follow-up and citizen-verified outcomes. This separation reduces the risk of attributing an operational result to an actor who does not control the work.

**[5] Kaylyn Jackson Schiff (2025)**

Open-data and transparency initiatives make public information more useful when figures are accompanied by provenance, update dates and a clear description of what each figure represents. CivicLens therefore records a source and verification state for project and expenditure data and explicitly displays “data unavailable” when a reliable value cannot be obtained.

**[6] William O’Brochta Kern,**

Digital grievance workflows involve multiple roles, so access control and auditability matter as much as interface design. Citizens should see their own case details, field workers should see only assigned work, officers should manage authorized departmental cases, and public viewers should see privacy-safe aggregates. These role boundaries inform the platform’s proposed permissions.

**[7] Rajesh Dinesh Hanbal et al.**

Recurring problems are often a signal that a one-time repair may not be sufficient. CivicLens groups similar reports by location and category to make recurrence visible to officers and constituency-level viewers. The initial prototype can use transparent rules and database queries; large-scale prediction models are outside the required first version.

**[8] Bokyong Shin et al.**

Trust depends on preventing a simple status change from silently erasing prior events. The complaint timeline retains assignment, work updates, resolution claims and citizen verification. Disputed cases can be reopened or manually reviewed, and one anonymous response should not automatically determine official closure.

### [9] Shirley Kempeneer and Johan Wolswinkel

The project's public-money feature is designed as a careful association rather than an automatic claim of causation. A recurring road complaint may be shown beside a related road project only when the location and available records support the link. The displayed amount, implementing agency and status must be traceable to an official source or labelled as sample data.

## 2.3 Comparative Analysis

**Table 2.1 Literature and civic-service comparison**

| Ref. | Author(s) / Year | Study focus | Methodology / approach | Relevance to CivicLens |
|---|---|---|---|---|
| [1] | Government of India / current portal | Public grievance redress | Centralized online grievance submission and tracking | Shows the importance of a traceable complaint record; CivicLens is not claimed to be integrated with CPGRAMS. |
| [2] | OpenStreetMap contributors | Open geographic data | Collaborative geographic data and map services | Can support map visualization, subject to data and jurisdiction validation. |
| [3] | Leaflet documentation | Interactive web maps | Open-source JavaScript mapping library | Supports category filters and map markers in the proposed interface. |
| [4] | World Bank GovTech resources | Digital public services | People-centred digital government approaches | Reinforces usability, institutional process and public-service delivery as design concerns. |
| [5] | United Nations SDG 11 | Sustainable communities | Global goals for inclusive, safe and resilient settlements | Provides context for civic infrastructure and local service visibility. |
| [6] | United Nations SDG 16 | Institutions and accountability | Global goals for effective, accountable and inclusive institutions | Aligns with transparent workflows and accessible public information. |
| [7] | Supabase documentation | Application data services | PostgreSQL, authentication and storage services | Offers a practical free-tier option for a student prototype. |

| Ref. | Author(s) / Year | Study focus | Methodology / approach | Relevance to CivicLens |
|---|---|---|---|---|
| [8] | React documentation | Web interface development | Component-based user-interface library | Supports reusable citizen and role-based dashboard interfaces. |
| | | | | |

## 2.4 Comparative Analysis of the Proposed System

CivicLens combines structured reporting, authority routing, timeline tracking, citizen verification, recurring-issue analytics and public project transparency. The comparative advantage is the integration of these stages in one workflow, with explicit privacy and accountability boundaries, rather than a claim that each individual capability is new.

**Table 2.2 Comparative analysis of civic-service approaches**

| Feature | Separate grievance channels | Public map/dashboard | CivicLens proposed platform | Design focus |
|---|---|---|---|---|
| Structured complaint reporting | Available in many channels | Limited | Yes | Clear categories, evidence and location |
| Authority routing visibility | Varies by service | Usually limited | Yes | Category + jurisdiction rules, manual review |
| Complaint timeline | Varies by service | Not primary purpose | Yes | Timestamped lifecycle and assigned authority |
| Citizen verification | Varies by service | No | Yes | Completely / partially / not resolved |
| Recurring issue view | Often fragmented | Yes, if data exists | Yes | Location clusters and category trends |
| Project/fund transparency | Separate sources | Can publish selected records | Yes, when sourced | Source, status and update date |
| Unified accountability workflow | Partial | No | Proposed integrated flow | Separate execution from representative oversight |
| | | | | |

## 2.5 Research Gap

A further gap concerns the relationship between operational records and public accountability. A public dashboard may display aggregate counts, but those numbers are useful only when users can understand their definitions, period and source. For example, “resolved” can mean that an officer marked work complete, whereas “citizen-verified” means a citizen supplied a positive response under the platform’s verification rules. These values should not be merged into a single number. Similarly, a recurring hotspot may indicate a maintenance issue, a reporting campaign or a location where people are more likely to submit reports; it should prompt investigation rather than be treated as proof of departmental failure. CivicLens addresses this interpretive gap by proposing separate status and outcome measures, source labels for public financial records, and explanations alongside analytics. The research contribution is the integration and careful framing of these elements in a coherent prototype workflow, not a claim that the platform has already improved civic service outcomes.

The reviewed concepts provide established building blocks, but a common user journey still needs to connect issue submission, responsible authority, field action, verification and public transparency. A portal can register a complaint without making the next action understandable; a map can show locations without explaining the resolution process; and an expenditure page can list amounts without connecting them to verified local outcomes.

CivicLens addresses this integration gap at prototype scale. It proposes a unified, auditable workflow and a source-aware public information layer. Its value must be assessed through usability testing, routing correctness, completion of the reporting journey, clarity of status information and the quality of the underlying data. The project does not claim that a dashboard alone guarantees better governance.

# CHAPTER 3

# THEORETICAL BACKGROUND AND SYSTEM DESIGN

## 3.1 General

CivicLens is a web-based information system that combines structured data capture, role-based workflows, geographic visualization and descriptive analytics. The system design emphasizes traceability: every complaint receives an identifier, important status changes become timeline events, and citizen verification is stored separately from an authority's resolution claim.

## 3.2 Digital Image Acquisition

A citizen creates a report by selecting a category and entering a short description. The user confirms the issue location and can attach supporting evidence. The interface validates required fields before submission, and the backend stores the report with a timestamp and unique identifier. Location information is used for jurisdiction lookup and map visualization; public views should generalize or hide details where needed to protect privacy.

Citizen interface → report form → validation → issue record and evidence metadata → authority-routing rules → complaint timeline

## 3.3 Image Preprocessing

Preprocessing includes trimming text, checking required fields, validating allowed categories and normalizing location data into the format used by the selected mapping component. Optional evidence is checked for permitted file type and size. The prototype should provide clear validation messages and should not reject a valid report solely because an optional field is empty.

## 3.4 Issue Classification and Routing

Issue classification begins with the category selected by the citizen and can be supported by keyword rules. The jurisdiction engine combines the category with the configured location boundary to suggest the department. The recommendation remains reviewable: if the boundary or authority is uncertain, an authorized officer or administrator can correct the assignment and the change is recorded.

### 3.5 Supabase / PostgreSQL

CivicLens uses an open and modular web architecture. React or Next.js can render the citizen interface and role-based dashboards, while server routes or a lightweight Node.js/FastAPI service handle validation and workflow operations. Supabase PostgreSQL can store users, issues, assignments, timeline events, verification records and projects. OpenStreetMap with Leaflet supports map views without requiring a paid map API.

The core data model separates the complaint from its evidence, assignment, timeline and verification events. This allows the application to preserve history when a complaint is reassigned or reopened. Public project records are stored separately from complaint records and linked only where supported by available data.

The platform should favor transparent rules over opaque automation in its first version. Category and jurisdiction mappings can be inspected by administrators, and uncertain routing should fall back to manual review. Optional AI can summarize a timeline or explain status terminology, but it must not invent official records or make final administrative decisions.

### 3.6 Issue Priority and Category Rules

A civic issue can be classified by category, severity, duration, number of people affected and proximity to essential facilities. In the first version, priority can be calculated with explicit rules and displayed with an explanation. Safety risks such as exposed electrical wires or open manholes may be marked urgent, while a recurring issue can be raised for review. The score supports triage; it does not replace an officer’s judgement.

Rule-based prioritization is explainable and inexpensive to implement. Each factor can be displayed to authorized users, and the rules can be tuned after stakeholder

feedback. The system should retain the original citizen report and record any manual priority override with the user, time and reason.

### 3.7 Jurisdiction Rule Engine

Authority routing must use a configured mapping between issue categories, administrative boundaries and responsible departments. OpenStreetMap and Leaflet can support location display, but map coordinates alone do not determine legal jurisdiction. The pilot configuration must be reviewed against authoritative administrative information, and uncertain assignments should be flagged for human review.

### 3.8 OpenStreetMap / Leaflet

OpenStreetMap-based mapping and Leaflet provide the map foundation for CivicLens. The map can display issue markers, filter by category and status, and cluster dense locations. The public layer should show privacy-safe summaries, while precise evidence and personal details remain available only to authorized users.

### 3.9 Map-Based Issue Tracking

The map view translates a stored issue location into a visual marker and associates it with category, status and date filters. Repeated reports near the same location can be grouped for review. Location accuracy depends on citizen confirmation and available boundary data, so the map should not be treated as definitive proof of department jurisdiction.

### 3.10 Location Mapping and Control

Issue-to-screen mapping places reports within the selected constituency or local-government area. Filters should let users compare open, resolved, overdue and recurring issues without presenting sample values as live government statistics. Where data is synthetic, the dashboard must label it clearly as demonstration data.

### 3.11 Evidence and Report Acquisition

The system receives structured information from the report form and optional image evidence from the citizen. Uploads should be associated with the complaint ID and

stored using access rules. Evidence supports review but does not automatically prove that a complaint is genuine or that a repair was completed.

### 3.12 Text Normalization

Text descriptions can be normalized by removing excess whitespace and checking required fields. Optional voice input may use browser speech recognition where supported, but typed reporting remains available. A transcript should be reviewed by the citizen before submission so that speech recognition errors do not silently alter the complaint.

### 3.13 Optional Civic Assistant

Natural-language processing is optional in the initial version. Simple keyword rules can suggest a category or summarize the complaint, but the selected category and final routing remain visible and correctable. If a model is introduced later, its output should be presented as a suggestion and evaluated against realistic civic examples.

The report text is converted into a consistent representation for search, duplicate hints and summaries. Normalization can include lowercasing, whitespace cleanup and common spelling variants, while preserving the original text for audit and review. Any automatic summary should remain linked to the source complaint.

### 3.14 rule-based classifications

A machine-learning model is not a mandatory dependency for the core CivicLens workflow. The project can demonstrate classification and duplicate hints through transparent rules and later evaluate an open-source model if suitable training data becomes available. No model should decide an official status or fabricate project information.

For optional classification, the input may be represented by category labels, keywords, location and timing features. Training data must be reviewed for quality and bias before a model is deployed. Until a validated model exists, deterministic rules and human review provide a safer baseline for a student prototype.

### 3.15 Issue Classification and Routing

Issue classification and routing translate a report into a suggested workflow destination. The system should show the selected category, likely authority and reason for the routing decision. Officers can reassign a case when jurisdiction is incorrect, and the audit trail retains both the original and revised assignment.

### 3.16 Complaint Workflow Layer

The workflow layer enforces role permissions and valid status transitions. Citizens create and track their reports; officers review, assign and update authorized cases; field workers submit progress evidence for assigned tasks; representative offices view constituency-level unresolved and recurring issues; administrators maintain categories, jurisdictions and source metadata. Every significant action records who performed it and when.

### 3.17 System Architecture

The presentation layer is responsible for the experience appropriate to each role. Citizens receive a concise reporting flow and a personal issue list; officers receive queues and filters relevant to their department; field workers see assigned tasks and the information needed to perform them; representative offices see constituency-level summaries and unresolved hotspots; administrators maintain categories, jurisdiction mappings and source records; and public viewers see published aggregate information. Authentication and authorization sit between the interface and protected services so that hiding a control in the browser is never the only access restriction. The service layer validates requests and applies business rules, while the data layer preserves the issue record and its related events. The analytics layer reads approved records and calculates descriptive indicators. This separation allows the team to develop the reporting workflow first and add public transparency features after the core records and permissions are reliable.

The architecture has six logical layers: presentation, authentication and role control, issue and evidence services, jurisdiction and workflow rules, data storage, and analytics/public transparency. Notifications and the optional Civic Assistant sit alongside the core workflow. The core application remains usable without paid AI, mapping or messaging services.

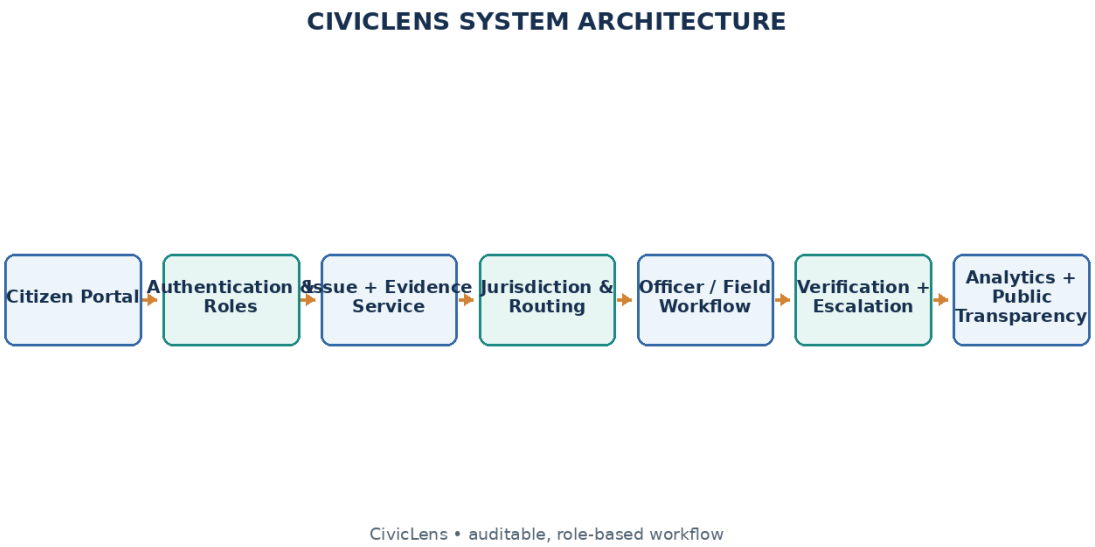



Figure 3.1 CivicLens system architecture

## 3.18 Data Flow Diagram

Data handling should follow a predictable sequence. The report service first validates the submitted fields and creates the issue record. Evidence metadata is associated with the same issue identifier, while uploaded files are stored under access-controlled paths. The routing service reads the category and confirmed location, checks the pilot's jurisdiction configuration and returns a suggested authority together with a routing status. An officer action creates an assignment event. Subsequent field updates and resolution claims are appended to the timeline. Citizen verification is stored as a separate event so that an authority's claim and a resident's response can both be reviewed. Analytics queries should use approved fields and a clearly defined reporting period. They should exclude private contact data and avoid counting duplicate records as separate resolved outcomes when a duplicate relationship has been confirmed. These data-flow rules help the team demonstrate not only the visible interface but also the consistency of the underlying records.

The data flow diagram describes the platform in terms of records and actions. The context diagram shows citizens, department officers, field workers, representative

offices and public viewers interacting with CivicLens. Lower-level diagrams expand the reporting, routing, assignment, verification and analytics processes.

### 3.18.1 Level 0

The context diagram treats CivicLens as one system and shows its external actors. Citizens submit reports and verification responses; officers receive assigned cases and return decisions or status updates; field workers submit work notes and evidence; representative offices view oversight information; and public viewers access privacy-safe dashboards and source-labelled project records. The system returns complaint IDs, timelines, notifications and aggregate information.

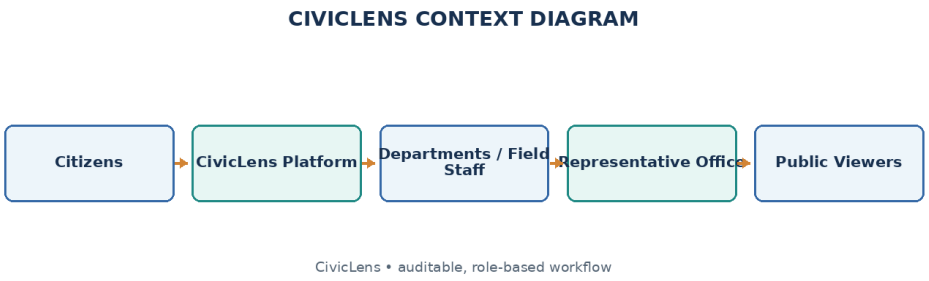



Figure 3.2 Data flow diagram – Level 0

### 3.18.2 Level 1

The first level decomposes CivicLens into report intake, evidence/location validation, authority routing, complaint workflow, field update handling, citizen verification and public analytics. The complaint database stores the main record, while separate timeline events preserve the history of actions. Project data remains a distinct domain and is associated only when a defensible link exists.

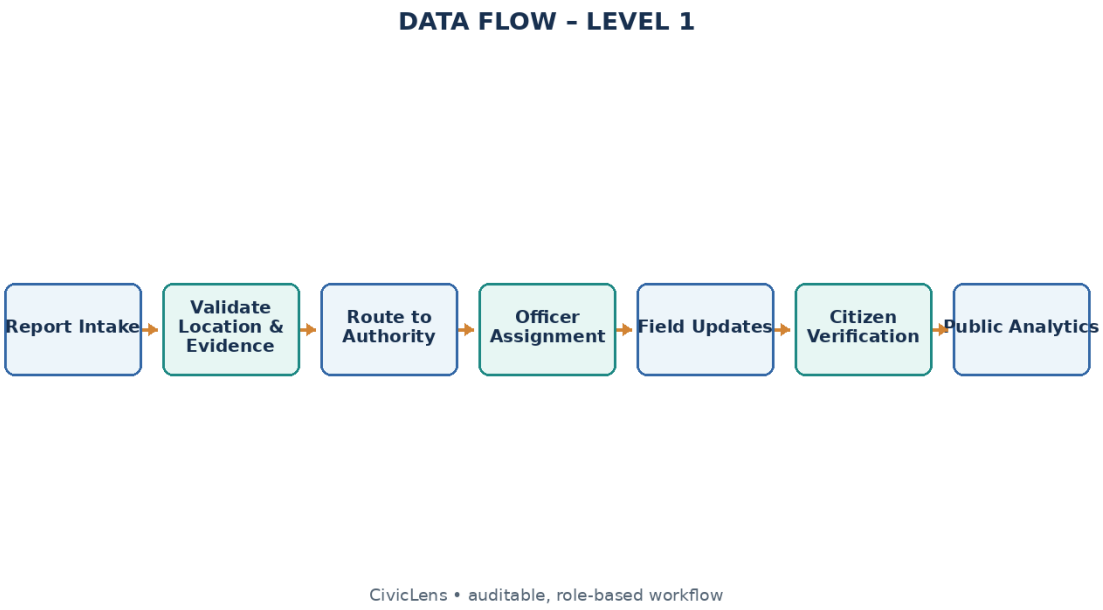



Figure 3.3 Data flow diagram – Level 1

### 3.18.3 Level 2

The second-level view expands the complaint workflow into status validation, assignment updates and event recording. A submitted report is checked for completeness, assigned a unique identifier and routed according to configured rules. An officer can accept, reassign or request clarification; a field worker records inspection or work evidence; and the resolution claim triggers a citizen verification request.

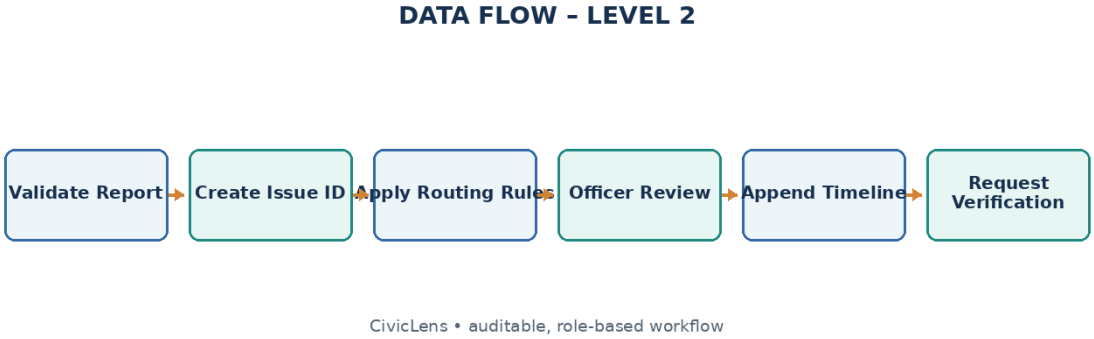



Figure 3.4 Data flow diagram – Level 2

## 3.19 Use Case Diagram

The primary actors are Citizen, Department Officer, Field Worker, Representative Office, Administrator and Public Viewer. Citizen use cases include submitting a report, tracking a complaint, attaching evidence and verifying a resolution. Officers manage assignments and status, field workers update assigned tasks, representative offices monitor aggregate constituency issues, administrators configure the system, and public viewers browse published information.

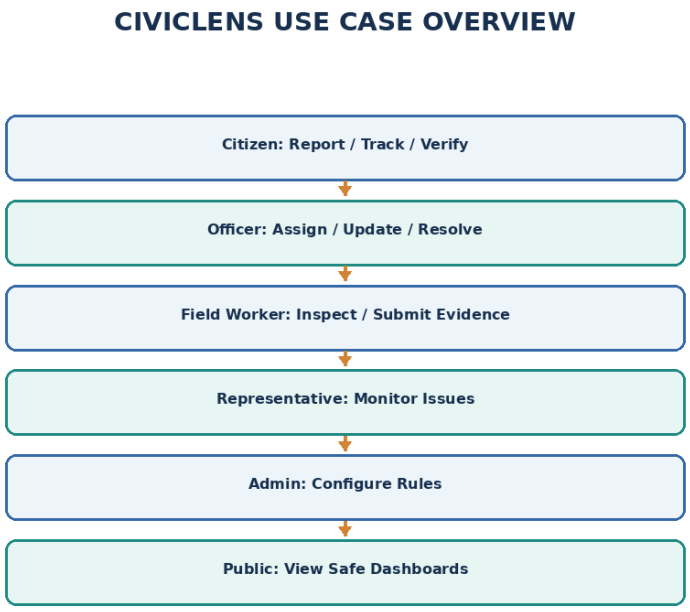



Figure 3.5 CivicLens use case diagram

## 3.20 Activity Diagram

The activity flow begins when a citizen submits a civic issue. The system validates required details, creates a complaint ID and applies routing rules. An officer reviews and assigns the case, a field worker records inspection and progress, and the officer may mark the work resolved. The citizen then verifies the outcome. A partial or negative response sends the case back for review, reopening or escalation; verified closure contributes to aggregate analytics.

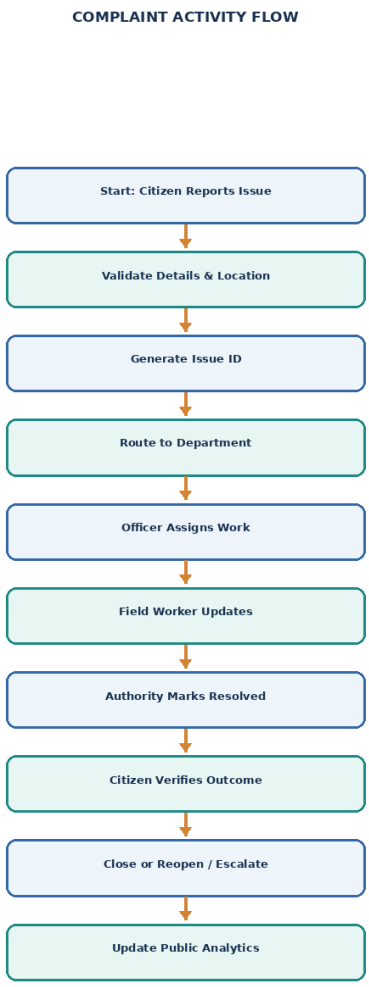



Figure 3.6 CivicLens activity diagram

## 3.21 Overall System Diagram

The overall system diagram presents the citizen, authority and public-information paths as parts of one platform. Citizen reporting creates the core issue record; officers and field workers progress the complaint; citizen verification closes or reopens the workflow; and the analytics layer produces maps and constituency summaries. Public project data is displayed with source and verification labels.

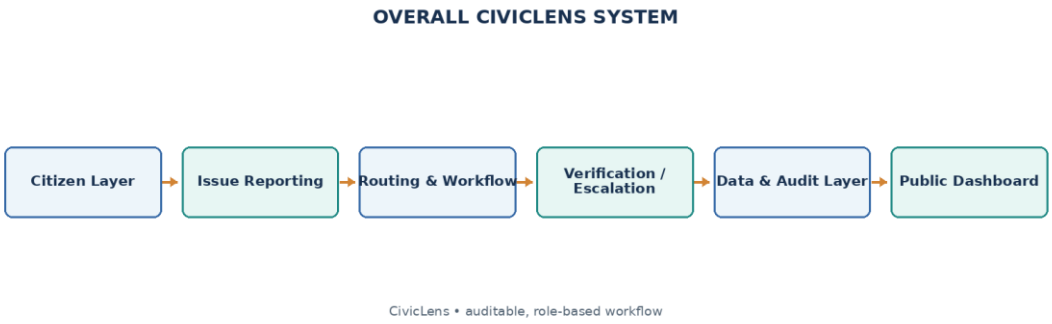



Figure 3.7 Overall CivicLens system diagram

## 3.22 Core Conceptual Classes

The issue record should contain the minimum structured fields needed for routing and follow-up: a unique identifier, reporter reference, category, title, description, location, created timestamp, current status and privacy setting. Evidence should be represented separately so that access rules can differ from the general complaint summary. Assignments should record the target department, assigned officer or team, assignment time and any reassignment reason. Timeline events should be append-oriented: a later status must not erase the previous assignment, field update or citizen response. Verification records should include the selected outcome, optional comment, timestamp and associated complaint. Escalation records should capture the trigger, destination, reason and review state. Public projects and financial records should remain separate entities, with source URL or document reference, reporting date, verification state and the fields actually available. This model avoids placing every concern in one oversized complaint table and makes auditability easier to demonstrate.

The conceptual model includes User/Profile, Issue, IssueEvidence, Authority/Department, Assignment, TimelineEvent, CitizenVerification, Escalation, Jurisdiction, Project, ProjectFinancialRecord, Notification and AuditLog. The entities are linked by complaint identifiers and role-controlled references. Table 3.1 summarizes representative classes and responsibilities.

**Table 3.1 Core CivicLens data entities**

| Class | Representative attributes | Representative methods |
|---|---|---|
| User / Profile | userId, role, contact preferences | register(), authenticate(), updateProfile() |
| Issue / Complaint | issueId, category, description, location, status | createIssue(), updateStatus(), getTimeline() |
| IssueEvidence | evidenceId, issueId, fileType, storagePath | attachEvidence(), validateEvidence() |
| Authority / Department | authorityId, categoryRules, jurisdictionId | routeIssue(), acceptAssignment(), reassign() |
| Assignment / TimelineEvent | assignmentId, actorId, timestamp, eventType | assignWork(), appendEvent() |
| CitizenVerification | verificationId, issueId, outcome, comments | submitVerification(), reopenIfRequired() |
| Project / FinancialRecord | projectId, agency, amount, source, status | validateSource(), updateProjectRecord() |

# CHAPTER 4

# METHODOLOGY AND PROJECT DESCRIPTION

## 4.1 General

The Phase I method followed a structured design sequence: understand citizen and authority needs, define the accountability gap, select the smallest useful feature set, design the architecture and data model, specify requirements, and plan prototype testing. The team's ideation sheet and CivicLens module specification provide the basis for the selected scope. The initial target is a single pilot geography and a limited set of high-frequency civic categories.

## 4.2 Requirement Analysis

The requirements also establish clear rules for what may be published. A public issue summary can include its category, generalized location, submission date and current public status, but should not expose a citizen's phone number, exact residential address or unreviewed attachments. Internal notes may contain information needed by an authorized officer, yet they should not automatically become public. The application should distinguish required and optional fields, provide clear error messages, and allow a citizen to correct a report before final submission. If a complaint is suspected to be a duplicate, the system can show a possible match and let an authorized user review it rather than silently discarding the new report. If the routing service cannot confidently identify a department, the complaint should remain in a visible review state instead of being assigned arbitrarily. These requirements reduce the chance that automation conceals uncertainty or prevents a citizen from obtaining help.

Requirements were derived from the reported pain points: uncertainty about responsibility, fragmented progress information, disputed resolution claims and limited visibility into recurring issues. The proposed system must collect sufficient information

for action without making reporting burdensome. It must also distinguish private case details from public aggregates and keep sample project data clearly labelled.

## 4.3 Functional Requirements

The main functional obligations are to authenticate users, accept structured civic reports, preserve evidence metadata, route complaints by category and jurisdiction, maintain a status timeline, support officer and field-worker actions, request citizen verification, reopen or escalate disputed cases, and publish privacy-safe analytics and project records.

**Table 4.1 Functional requirements**

| ID | Requirement |
|---|---|
| FR1 | The system shall authenticate users and apply permissions according to the assigned role. |
| FR2 | The system shall accept a civic report with category, title, description and confirmed location. |
| FR3 | The system shall allow optional photo/video evidence and store evidence metadata against the issue ID. |
| FR4 | The system shall generate a unique complaint identifier and an initial status. |
| FR5 | The system shall suggest a responsible authority using category and configured jurisdiction rules. |
| FR6 | The system shall allow authorized officers to accept, reassign or request clarification. |
| FR7 | The system shall maintain a timestamped timeline for assignment, work updates and status changes. |
| FR8 | The system shall allow field workers to submit inspection notes and progress/completion evidence. |
| FR9 | The system shall request citizen verification when an authority marks a complaint resolved. |
| FR10 | The system shall support reopening or escalation when a citizen reports partial or unsuccessful resolution. |
| FR11 | The system shall present privacy-safe maps and constituency analytics from stored records. |
| FR12 | The system shall display public project and expenditure information only with source/verification labels or sample-data labels. |
| FR13 | The system shall record important actions in an audit trail and restrict access to private records. |

## 4.4 Non-Functional Requirements

Non-functional requirements include usability, accessibility, privacy, traceability, reliability, maintainability and affordability. The system should be responsive on mobile devices, work with free-tier services or local deployment, explain routing

decisions and maintain an audit trail for important actions. Public metrics must identify their period, source and limitations.

**Table 4.2 Non-functional requirements**

| Requirement | Description |
|---|---|
| Usability | Clear categories, minimal typing, mobile-friendly reporting and understandable status labels. |
| Accessibility | Readable controls, Tamil/English examples and voice-assisted input where supported. |
| Privacy | Hide personal contact details and sensitive evidence from public views. |
| Traceability | Timestamp important actions and retain assignment, verification and escalation history. |
| Reliability | Validate required fields and preserve complaint history across reassignment and reopening. |
| Explainability | Show routing rationale and make priority rules inspectable. |
| Affordability | Avoid paid APIs and services as mandatory dependencies. |
| Maintainability | Separate reporting, routing, workflow, verification and analytics modules. |

## 4.5 Technology Selection

The selected tools are recommendations for the intended prototype rather than a claim that every service has already been deployed. The final report should record the actual versions, environment variables, database policies and deployment configuration used by the team. Secrets such as database service keys must remain on the server and must never be embedded in public client code or committed to the repository. Database access policies should be tested with separate citizen, officer, field-worker and public-viewer accounts. If hosted services are unavailable during a demonstration, a controlled local dataset can be used, provided it is clearly labelled and the demonstration does not imply a live connection to a government system. The project should keep its core complaint workflow functional even if the optional assistant or external notification mechanism is disabled.

Technology choices are guided by the free-only project requirement and the need for a demonstrable web prototype. React or Next.js with Tailwind CSS supports the interface; Supabase PostgreSQL and Auth provide a convenient data and access layer; OpenStreetMap with Leaflet supports maps; and Recharts or Chart.js can visualize

aggregate records. Rule-based fallbacks keep the core journey independent of paid AI APIs.

**Table 4.3 Technology stack**

| Component | Technology | Role |
|---|---|---|
| Frontend | React / Next.js + Tailwind CSS | Responsive citizen portal and role-based dashboards. |
| Database | Supabase PostgreSQL / PostgreSQL | Store users, issues, assignments, timeline, verification and project records. |
| Authentication | Supabase Auth or equivalent | Sign-in and role-controlled access. |
| File storage | Supabase Storage or local demo storage | Store optional issue evidence with access policies. |
| Maps | OpenStreetMap + Leaflet | Show issue markers, filters and recurring locations. |
| Analytics | Recharts / Chart.js + database queries | Visualize category counts, status and recurring trends. |
| AI fallback | Rule-based logic; optional open-source model | Explain status and suggest categories without mandatory paid APIs. |
| | | |

Table 4.4 summarizes why each selected technology fits the prototype constraints. The stack is illustrative and can be adapted to the team's actual implementation, provided the report clearly distinguishes proposed components from components already integrated.

**Table 4.4 Technology selection rationale**

| Technology | Reason for selection |
|---|---|
| React / Next.js | Supports reusable, responsive interfaces and can combine frontend routes with server-side functions. |
| Tailwind CSS | Enables consistent responsive styling with a small design system. |
| Supabase PostgreSQL | Provides relational storage suitable for connected complaint, assignment and audit records. |
| Supabase Auth | Offers a practical authentication option for a student prototype. |
| OpenStreetMap + Leaflet | Supports map-based issue views without requiring a paid mapping API. |
| Recharts / Chart.js | Provides charts for status, category and recurring-issue summaries. |
| Rule-based routing | Is explainable, easy to test and can be corrected by authorized officers. |

## 4.6 System Modules

CivicLens is organized into connected modules rather than separate mini-projects. Authentication establishes role boundaries; issue reporting captures the complaint; routing and tracking coordinate action; verification and escalation preserve citizen feedback; officer and field-worker dashboards support operational work; and public maps, analytics and project transparency expose aggregate information. Notifications and the Civic Assistant enhance the core journey but are not prerequisites.

**Table 4.5 System module descriptions**

| Module | Description |
|---|---|
| Authentication & Role Management | Controls access for citizens, officers, field workers, representative offices, administrators and public viewers. |
| Civic Issue Reporting | Captures category, description, location, impact and optional evidence; returns a unique issue ID. |
| Location & Authority Routing | Uses category and configured jurisdiction rules to suggest the responsible authority, with manual review. |
| Complaint Tracking & Timeline | Records submission, assignment, acknowledgement, work updates, resolution and verification events. |

### 4.6.1 Civic Issue Reporting Module

The reporting module collects category, title, description, location, impact and optional evidence. Required fields are validated before submission and the user receives a unique complaint ID. The interface should be mobile-friendly, minimize typing and allow the citizen to review the report before it is sent.

### 4.6.2 Authority Routing and Complaint Tracking Module

The authority and jurisdiction module combines the issue category with the configured pilot-area rules to suggest a responsible department. It displays the routing result and allows authorized review or reassignment when the mapping is uncertain. Changes are recorded so that the original routing decision remains traceable.

### 4.6.3 Citizen Verification and Transparency Module

The workflow and assistance module presents the complaint timeline, status definitions and next expected step. A simple rule-based assistant can explain how to report a problem, interpret status terms or summarize a case. Optional AI must remain grounded in stored records and must not make administrative decisions.

### 4.6.4 Workflow and Analytics Module

The verification and transparency module records the citizen's response after an authority marks a case resolved. Partial or negative verification can trigger review, reopening or escalation. Aggregated records feed public maps and dashboards, while project and expenditure details are shown only with source and verification information.

## 4.7 Complaint Status and Action Mapping

Each report follows an explicit status path such as Submitted, Under Verification, Assigned, Acknowledged, In Progress, Resolved, Awaiting Citizen Verification, Reopened, Escalated or Closed. Status transitions are controlled by role permissions and logged with timestamps. A claimed resolution is not silently treated as citizen-verified closure.

**Table 4.6 Mapping of user actions to workflow outcomes**

| User / event | Recognized input | System action |
|---|---|---|
| Citizen report | Category, description and location | Create issue ID and submit report |
| Evidence attachment | Photo/video selected by citizen | Validate and associate evidence with issue |
| Routing engine | Issue category + jurisdiction | Suggest department and record routing rationale |
| Department officer | Accept / reassign / request clarification | Update assignment and append timeline event |
| Field worker | Inspection note / progress photo | Record work update and evidence |
| Authority resolution claim | Mark work resolved | Set awaiting citizen verification and notify citizen |
| Citizen verification | Completely / Partially / Not Resolved | Close after verification or reopen/escalate for review |
| Escalation rule | Overdue or disputed case | Route to authorized oversight layer and preserve history |
| Public dashboard | Approved aggregate issue records | Update map, category/status metrics and recurring hotspots |
| Project transparency | Source-verified or labelled sample record | Display project status, funding details and provenance |
| Public viewer | Filter by area, category, status and period | Show privacy-safe published information |
| Administrator | Category, jurisdiction or role change | Update configuration with audit record |
| Civic Assistant | Question about process/status | Provide grounded rule-based explanation from available information |
| Data unavailable | No verified financial/project source | Display unavailable rather than fabricate a value |

| User / event | Recognized input | System action |
|---|---|---|
| | | |

## 4.8 Module Integration

A typical end-to-end scenario illustrates how the modules interact. A resident reports a pothole, selects Roads, confirms the map location and attaches a photograph. The system validates required fields and creates an issue identifier. The jurisdiction rules suggest the relevant roads authority and show that routing decision in the issue details. An authorized officer reviews the report, accepts or corrects the assignment and allocates a field task. The field worker records an inspection note and later adds a completion update. The officer marks the work resolved, which moves the complaint into an awaiting-verification state rather than directly hiding it from the citizen’s list. The resident can confirm the result or select partially resolved or not resolved. A disputed case remains visible to authorized reviewers and can be reopened or escalated under the configured rules. Only the resulting approved records are aggregated into public dashboards. This scenario is also a useful integration test because it exercises identity, role permissions, routing, evidence metadata, timeline history, verification and analytics together.

Integration depends on shared complaint identifiers and consistent role permissions. The reporting module creates the core record; routing stores the suggested and confirmed authority; officer and field-worker actions append timeline events; verification updates the case state; and analytics read approved fields from the underlying records. Public project records remain separate until a source-supported association is entered.

Two integration concerns require special care. First, routing can be wrong when boundaries or department responsibilities are incomplete, so manual correction and audit history are necessary. Second, citizen verification may conflict with an authority’s resolution claim, so the platform must preserve both events and route disputes for review instead of overwriting one side.

## 4.9 System Configuration

For demonstration purposes, each role should be tested with a separate account and a defined set of permissions. A citizen should not be able to edit another person's complaint or view private attachments belonging to an unrelated case. A field worker should only see assigned tasks and should not change jurisdiction rules. An officer should manage records within the configured department scope, while a representative-office account should see oversight summaries without being able to claim that the department's work has been completed. An administrator can configure categories and jurisdiction mappings, but important configuration changes should still be logged. A public viewer should have read-only access to published summaries and source-labelled project information. These role distinctions should be reflected in the database policies and server-side checks, not just in navigation menus. During evaluation, the team should demonstrate both an allowed action and a denied action for each sensitive role boundary. This provides a simple, understandable proof that the prototype treats privacy and accountability as system requirements rather than visual design choices.

The report describes a free-first configuration suitable for a student prototype. Hardware needs are limited to an internet-enabled computer and a smartphone or camera for collecting demonstration evidence. The software configuration may use a modern browser, Node.js or Python backend, PostgreSQL/Supabase, Git and GitHub. Actual versions should be recorded after the implementation environment is finalized.

**Table 4.7 Hardware configuration**

| Component | Specification |
|---|---|
| Processor | Standard laptop/desktop capable of running a modern browser |
| RAM | 4 GB minimum recommended for local development; more may improve comfort |
| Storage | Sufficient space for source code, database tools and sample evidence |
| Internet | Required for hosted development/deployment; local demonstration can use local services |
| Camera / phone | Optional device for collecting demonstration photos of civic issues |
| Display | Desktop or mobile screen for responsive-interface testing |

**Table 4.8 Software configuration**

| Component | Specification |
|---|---|
| Operating system | Windows, Linux or macOS |
| Frontend | React / Next.js and Tailwind CSS |
| Backend | Next.js server routes or Node.js / FastAPI |
| Database / authentication | Supabase PostgreSQL and Auth, or local PostgreSQL |

## 4.10 Feasibility Study

The scope can be controlled through a staged implementation. The first milestone is a working citizen report form, database schema and unique complaint ID. The second adds configured authority routing and the officer's assignment queue. The third connects field-worker updates to the timeline and adds citizen verification. The fourth introduces public maps and analytics, followed by source-labelled project records. This order is important because the public dashboard depends on consistent and trustworthy underlying records. Building advanced visualizations before the complaint lifecycle is stable would create a polished interface without reliable information. A single pilot geography and a small set of categories also make it easier to validate routing rules, create realistic demonstration records and explain limitations during evaluation. The project should use synthetic or approved sample data where real records are unavailable and should never present seeded values as live government statistics.

Feasibility was considered across cost, technical implementation and social adoption. The core functions can be demonstrated with widely available web technologies and a limited pilot dataset. The most important constraints are not expensive hardware but accurate jurisdiction configuration, realistic workflow design, data quality, privacy and stakeholder validation.

### 4.10.1 Economical Feasibility

The economic feasibility is favorable because the recommended stack uses open-source libraries and free service tiers for a limited prototype. Costs may arise if usage exceeds free limits, if paid notifications are introduced or if production hosting is required. The design therefore avoids paid services as mandatory dependencies and supports local demonstration with labelled sample data.

### 4.10.2 Technical Feasibility

The technical scope is achievable using common web development tools. The team can implement the reporting form, role-based dashboards, database entities, rule-based routing and status transitions incrementally. Mapping and analytics can be added after the complaint workflow is stable. Official datasets and jurisdiction boundaries may require additional validation before the system can be used outside a controlled demonstration.

### 4.10.3 Social Feasibility

Social feasibility depends on whether citizens understand the categories, status labels and verification step. Tamil and English examples, readable controls, minimal typing and optional assisted reporting can reduce barriers. Department officers and field workers need concise work queues, while representative offices should receive oversight information without being represented as the executing authority.

## 4.11 Testing Methodology

A practical acceptance dataset should include ordinary and exceptional scenarios. Ordinary cases can cover a road defect, uncollected garbage, drainage overflow, water-supply interruption and a broken streetlight. Exceptional cases should include a report with missing location, a possible duplicate, an issue near a jurisdiction boundary, a department reassignment, a long-pending complaint, a citizen who rejects a resolution claim and a project record without a verifiable financial source. For each case, the expected result should be documented before execution. Test evidence can include screenshots, database records and the event timeline, but personal details should be masked in material used for public demonstrations. Defects should be logged with severity and steps to reproduce. A successful visual demonstration is not sufficient on its own: the team should also confirm that the underlying status and audit records are correct, that unauthorized roles cannot change protected data, and that public totals match the records included in the selected reporting period.

Testing should be planned at module, integration, functional and system levels. Because the supplied documents describe the concept and selected modules rather than verified live government integration, the report must not claim production validation.

Tests should use clearly labelled sample records and confirm that role permissions, routing, status history, citizen verification and privacy boundaries behave as specified.

### 4.11.1 Unit Testing

Unit testing can validate category checks, complaint-ID generation, jurisdiction lookup, status-transition rules and data formatting independently. Fixed sample records should cover valid submissions, missing fields, unsupported categories and uncertain routing. A unit test passes only when the observed output matches its documented expectation.

### 4.11.2 Integration Testing

Integration testing should focus on module boundaries: report creation to routing, routing to officer assignment, field updates to the timeline, resolution claims to citizen verification, and approved records to public analytics. It should also confirm that private evidence and contact information do not leak into public endpoints.

### 4.11.3 Functional Testing

Functional testing checks each requirement from the user's perspective. A citizen should be able to submit and track a report; an officer should be able to assign and update it; a field worker should be able to attach progress evidence; and a citizen should be able to confirm or dispute a claimed resolution. Public dashboard figures should match the underlying demonstration records.

### 4.11.4 System Testing

System testing exercises the complete journey from report creation through assignment, field update, resolution claim, citizen verification and dashboard aggregation. Additional scenarios should include an incorrect authority assignment, a duplicate report, an overdue case, a disputed closure and unavailable project data.

### 4.11.5 White Box and Black Box Testing

White-box testing examines routing rules, role checks, status transitions and data transformations. Black-box testing submits realistic civic scenarios through the user interface and verifies the displayed result. Security and privacy checks should confirm

that users cannot modify records outside their permissions and that public pages hide personal details.

The test cases in Table 4.9 define the expected behavior for the proposed prototype. Results should be marked Pass only after execution in the actual build; until then, they are acceptance criteria rather than evidence of completed testing.

**Table 4.9 Prototype acceptance test cases**

| Test | Input | Expected output | Result |
|---|---|---|---|
| Report validation | Report missing required category or location | Clear validation message; no incomplete issue created | To be tested |
| Issue creation | Valid report with category and location | Unique issue ID and initial status displayed | To be tested |
| Authority routing | Road complaint in configured pilot area | Mapped authority suggested with routing status | To be tested |
| Manual reassignment | Officer changes suggested authority | New assignment recorded; prior event retained | To be tested |
| Field update | Assigned field worker submits note/evidence | Timeline updated with actor and timestamp | To be tested |
| Citizen verification | Authority marks case resolved | Citizen receives Completely / Partially / Not Resolved choices | To be tested |
| Disputed resolution | Citizen selects Not Resolved | Case reopens or enters authorized review path | To be tested |
| Escalation | Case exceeds configured response timeline | Escalation event recorded and visible to authorized roles | To be tested |
| Public privacy | Open public issue map | No private phone number, address or sensitive evidence exposed | To be tested |
| Project source | Project record has no reliable source | Data shown as unavailable or labelled sample, never fabricated | To be tested |
| Analytics consistency | Known sample complaint dataset | Dashboard counts match approved underlying records | To be tested |
| Role permissions | User attempts action outside assigned role | Action denied and unauthorized access prevented | To be tested |
| Recurring issue | Several similar reports in one location | Location/category hotspot identified for review | To be tested |

## 4.12 Initial Results

The Phase I outcome is a defined CivicLens concept, a reduced twelve-module scope, a role model, an end-to-end workflow, a proposed free-first architecture and a testable set of functional requirements. The design connects citizen reporting with authority routing, complaint timelines, field work, citizen verification, escalation, recurring-issue analytics and public project transparency.

The system is intended to demonstrate how the core workflow can be implemented using a modest web stack. Its success should be evaluated through routing correctness, successful completion of the report journey, clarity of timeline information, proper handling of disputed closures and consistency between public metrics and the source records. No live government integration or measured impact is claimed by this report.

## 4.13 Challenges

1. Data quality: incomplete descriptions, inaccurate locations and duplicate reports can reduce the usefulness of routing and analytics.
2. Jurisdiction mapping: administrative boundaries and department responsibilities must be configured for the pilot area and reviewed by a knowledgeable source.
3. Verification disputes: authority resolution claims and citizen feedback may conflict and require an auditable review path.
4. Privacy: images, contact information and precise locations must be protected through role-based access and careful public display.
5. Public-data provenance: project budgets and expenditure values must be traceable, dated and labelled as official, authority-submitted, citizen-reported or unavailable.
6. Adoption: citizens and officers may not use the workflow unless the interface is simple and the operational benefit is clear.
7. Scope control: advanced prediction, broad integrations and multi-constituency deployment should not delay a stable core prototype.
8. Accountability fairness: department execution and representative oversight must be measured separately, with clear definitions and limitations.

# CHAPTER 5

# CONCLUSIONS AND FUTURE ENHANCEMENTS FOR PHASE II

## 5.1 Conclusion

CivicLens proposes a citizen-centric civic accountability platform that connects local issue reporting with authority routing, transparent tracking, field updates, citizen verification, escalation and public information. It goes beyond complaint registration by preserving the full lifecycle and showing how recurring issues are distributed across a constituency. A separate project-transparency layer can connect civic problems to relevant development projects and expenditure records when reliable data is available.

The Phase I work establishes the problem definition, design-thinking framing, literature foundation, user roles, system architecture, data model, functional and non-functional requirements, technology choices, module specifications, workflow diagrams and testing plan. The project prioritizes a manageable free-first prototype and explicitly separates proposed features from validated implementation. Public metrics are intended to be evidence-based, privacy-safe and transparent about their sources and limitations.

## 5.2 Current Phase-I Status

1. Civic problem and user needs defined from the supplied ideation and concept documents.
2. Twelve core modules selected and grouped into one connected reporting-to-accountability workflow.
3. User roles and access boundaries specified for citizens, officers, field workers, representative offices, administrators and public viewers.
4. Complaint reporting fields, unique ID and evidence-handling requirements documented.

5. Category- and jurisdiction-based authority routing with manual review and audit history specified.
6. Complaint timeline, officer assignment and field-worker update flow documented.
7. Citizen verification, reopening and escalation behavior defined.
8. Public civic map, recurring-issue analytics and constituency dashboard scope established.
9. Project and expenditure transparency fields, source labels and sample-data safeguards documented.
10. Free-first technology stack and optional AI/rule-based fallback specified.
11. System architecture, data entities, role model and end-to-end workflow described.
12. Acceptance test scenarios prepared for reporting, routing, status updates, verification, privacy and analytics.
13. Prototype execution, stakeholder validation and quantitative usability evaluation remain future work unless separately evidenced.

## 5.3 Limitations

The proposed platform cannot independently guarantee that a reported issue is genuine, that a department's jurisdiction mapping is correct or that a public project record is complete. Photo evidence can be outdated or unrelated, a location can be imprecise, and a citizen may not respond to a verification request. The system therefore supports evidence review and administrative judgement rather than replacing them. The prototype also depends on the availability of public project information and cannot infer actual expenditure from a complaint or a map marker. Performance indicators require defined periods, denominators and exclusion rules; without those definitions, comparisons between areas or departments may be misleading. These limitations should be shown during the project presentation and revisited during pilot evaluation.

The report describes a design and prototype specification, not a verified production service. Routing depends on correct pilot geography and department mappings; map

markers depend on location accuracy; analytics depend on the quality and completeness of stored records; and public project transparency depends on reliable source data. The initial system also excludes broad government integrations and advanced predictive models. These limits should be made explicit during demonstration.

## 5.4 Phase-II Objectives

The next phase should produce evidence for each major claim made in the demonstration. A routing test should record the expected authority, actual suggestion and whether manual correction was needed. A timeline test should confirm that events appear in chronological order and that reassignment does not erase earlier actions. A verification test should demonstrate all three citizen outcomes and confirm that a disputed case remains available for review. A privacy test should inspect public pages and API responses, not only the visible interface. An analytics test should compare dashboard totals with a known set of sample records and document the rules for duplicates, reopened cases and reporting periods. Finally, user evaluation should ask participants to complete realistic tasks and describe whether they understand the responsible authority, the latest action and the next expected step. The results can guide changes to labels, navigation and workflow before the project is expanded to more categories or areas.

1. Complete the citizen reporting and authentication flow with a working database and unique complaint identifiers.
2. Implement jurisdiction-aware routing, officer assignment and a timestamped complaint timeline.
3. Build field-worker task updates with notes and optional evidence upload.
4. Implement citizen verification with completely resolved, partially resolved and not resolved outcomes.
5. Add reopening, escalation and overdue-case handling with preserved history.
6. Build a public civic map with category, status and date filters.
7. Create constituency dashboards with recurring-location and status analytics based on stored records.

8. Implement the public project module with source metadata and clearly labelled sample data.
9. Add in-app notifications for submission, assignment, status changes, verification requests and escalation.
10. Introduce an optional Civic Assistant using rule-based responses first, with model assistance only if safely grounded in stored records.
11. Conduct usability tests with citizens and role-based test accounts using realistic scenarios.
12. Validate the jurisdiction configuration against authoritative administrative information for the chosen pilot area.
13. Test role permissions, audit logs, privacy boundaries and public-data consistency.
14. Refine Tamil/English accessibility, mobile layout, readable status labels and minimal-typing flows.
15. Measure reporting completion, routing correction rate, timeline comprehension and citizen-verification completion in a controlled evaluation.
16. Evaluate recurring-issue grouping and duplicate hints against a labelled sample dataset.
17. Prepare deployment documentation, a demonstration dataset and final project report with limitations clearly stated.

## 5.5 Future Enhancements

Future evaluation should focus on whether the platform makes the process clearer and more usable, rather than assuming that digitization automatically improves governance. Suggested measures include the percentage of users who can submit a complete report without assistance, the percentage who correctly identify the responsible department after seeing the routing result, the time needed to locate the latest complaint update, the proportion of resolution claims receiving citizen feedback, and the frequency with which routing needs manual correction. Dashboard evaluation should check whether viewers understand the difference between submitted, authority-marked-resolved and citizen-verified cases. The public-money feature should be

reviewed for source completeness and for the risk of implying a financial relationship that the available records do not support. A small usability study can combine task completion, observation and participant feedback. These measures would provide a more defensible basis for refining CivicLens than relying only on feature counts or visual polish.

Future work is grouped into three areas. The first completes the core workflow: role-based access, reporting, routing, status tracking, field updates, citizen verification and escalation. The second strengthens transparency through maps, recurring-issue analytics, source-labelled project records and privacy-safe dashboards. The third adds optional enhancements such as notifications, Tamil/English support and a rule-based Civic Assistant. Any AI-assisted classification or summarization should be evaluated against labelled examples and should never replace authorized administrative decisions.

## 5.6 Phase-II Work Schedule

**Table 5.1 Future enhancements**

| Phase | Planned activity | Expected outcome |
|---|---|---|
| Phase II – Core workflow | Complete authentication, reporting, unique IDs, database and status timeline. | Working end-to-end complaint creation and tracking. |
| Phase II – Routing and assignments | Configure pilot jurisdiction rules, officer work queues and field-worker updates. | Traceable ownership and field progress. |
| Phase II – Verification | Implement citizen resolution choices, reopening and escalation. | Disputed closures preserved for review. |
| Phase II – Public transparency | Build issue map, recurring analytics and constituency dashboard. | Privacy-safe view of civic patterns. |
| Phase II – Project records | Add source-labelled public project and expenditure records. | Transparent provenance and explicit unavailable values. |
| Phase II – Accessibility | Refine mobile layouts, Tamil/English content and low-typing flows. | More inclusive reporting experience. |
| Phase II – Notifications | Add in-app updates for assignment, status changes and verification requests. | Users can follow important events. |
| Phase II – Validation | Test role permissions, routing, privacy, usability and data consistency. | Evidence for prototype quality and limitations. |
| | | |

# REFERENCES

[1] Government of India. Centralized Public Grievance Redress and Monitoring System (CPGRAMS). Official portal and process information. https://pgportal.gov.in/

[2] OpenStreetMap contributors. OpenStreetMap: collaborative geographic data and mapping project. https://www.openstreetmap.org/

[3] Leaflet. Leaflet JavaScript library for interactive maps. https://leafletjs.com/

[4] Supabase. Documentation: PostgreSQL database, authentication and storage. https://supabase.com/docs

[5] United Nations. Sustainable Development Goal 11: Sustainable Cities and Communities. https://sdgs.un.org/goals/goal11

[6] United Nations. Sustainable Development Goal 16: Peace, Justice and Strong Institutions. https://sdgs.un.org/goals/goal16

[7] World Bank. GovTech: putting people first in public service delivery. World Bank GovTech resources. https://www.worldbank.org/en/programs/govtech

[8] React. React documentation. https://react.dev/

[9] Next.js. Next.js documentation. https://nextjs.org/docs

# CO–SDG RELEVANCE RECORD

Project Title : CIVICLENS – CONSTITUENCY ACCOUNTABILITY & CIVIC TRANSPARENCY

Batch Members : Durgaprasad V (241701014)

Sanjaykumar R (241701049)

Supervisor : Dr. P. REVATHY, Ph.D.

| SDG (No. & Theme) | Addressed COs | Topic/Activity addressing SDG Theme |
|---|---|---|
| SDG 9 – Industry, Innovation and Infrastructure | CO2, CO4 | Uses web technologies, mapping and structured data to make civic service information more accessible. |
| SDG 11 – Sustainable Cities and Communities | CO1, CO3, CO4 | Helps residents report local infrastructure and service issues and observe recurring problem locations. |
| SDG 16 – Peace, Justice and Strong Institutions | CO1, CO3, CO5 | Supports transparent workflows, traceable actions, citizen verification and accountable public information. |
| SDG 17 – Partnerships for the Goals | CO2, CO4, CO5 | Encourages structured information exchange between citizens, service departments and public oversight roles. |

Signature of the Supervisor

**COURSE OUTCOMES**

CO1: Analyse civic-service delivery problems and define requirements for a transparent, citizen-centred complaint lifecycle.

CO2: Design a web-based reporting and authority-routing workflow using structured data, jurisdiction rules and role-based access.

CO3: Develop traceable complaint tracking, field-worker updates, citizen verification and escalation processes.

CO4: Apply geographic visualization and descriptive analytics to identify recurring civic issues and present constituency-level information.

CO5: Evaluate usability, data quality, privacy, fairness and transparency, and identify limitations and future validation work.

## PROGRAM OUTCOMES

| PO | Outcome | Description |
|---|---|---|
| PO1 | Engineering Knowledge | Apply computing and software-engineering knowledge to design a structured civic information system. |
| PO2 | Problem Analysis | Analyse reporting, routing, verification and transparency gaps using stakeholder needs and documented requirements. |
| PO3 | Design / Development of Solutions | Design role-based workflows, complaint lifecycle and dashboards that address civic-service needs. |
| PO4 | Conduct Investigations of Complex Problems | Plan tests for routing, privacy, status transitions and consistency of analytics using controlled sample data. |
| PO5 | Modern Tool Usage | Use modern web frameworks, databases, mapping libraries and version-control tools while recognizing their limits. |
| PO6 | The Engineer and Society | Consider citizen access, public accountability, privacy and fairness in the handling of civic reports. |
| PO7 | Environment and Sustainability | Support visibility of infrastructure and environmental-service issues such as drainage, waste and public facilities. |
| PO8 | Ethics | Use source provenance, role permissions, audit history and non-partisan reporting principles. |
| PO9 | Individual and Team Work | Collaborate across interface design, backend workflow, data configuration, testing and documentation. |
| PO10 | Communication | Present technical decisions, complaint status and public information in clear language. |
| PO11 | Project Management and Finance | Prioritize a limited pilot scope and plan implementation phases within free-tier constraints. |
| PO12 | Life-long Learning | Evaluate new civic technology, accessibility practices and open-data methods as the project evolves. |

## PROGRAM SPECIFIC OUTCOMES

| PSO | Area | Description |
|---|---|---|
| PSO1 | Computing Foundations | Apply programming, database and software design techniques to implement connected civic workflow modules. |
| PSO2 | HCI and Digital Experience Design | Design accessible citizen reporting and role-based dashboards around user needs. |
| PSO3 | Research, Innovation and Professional Growth | Identify a civic-service design gap and evaluate a prototype using structured requirements and tests. |
| PSO4 | Inclusive and Accessible System Design | Use readable, mobile-friendly, privacy-aware interfaces and simple reporting flows for diverse users. |

# CO–PO–PSO MAPPING

Project Title : CIVICLENS – CONSTITUENCY ACCOUNTABILITY & CIVIC TRANSPARENCY

Batch Members : Durgaprasad V (241701014)

Sanjaykumar R (241701049)

Supervisor : Dr. P. REVATHY, Ph.D.

| CO | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 | PSO1 | PSO2 | PSO3 | PSO4 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **CO1** | 3 | 3 | 3 | 3 | 3 | 3 | 2 | 2 | -- | 2 | -- | 2 | 3 | 2 | -- | -- |
| **CO2** | 3 | 3 | 3 | 3 | 3 | 3 | 2 | 2 | -- | 2 | -- | 2 | 3 | 2 | -- | -- |
| **CO3** | 3 | 3 | 3 | 3 | 3 | 3 | 2 | 2 | 3 | -- | 3 | -- | 3 | 2 | -- | -- |
| **CO4** | -- | -- | -- | -- | -- | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 2 | 3 | -- | -- |
| **CO5** | -- | 2 | 2 | -- | 2 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 2 | 3 | -- | -- |

1 – Slight (Low); 2 – Moderate (Medium); 3 – Substantial (High). If there is no correlation, put "--".

Signature of the Supervisor